Предсказание возраста по фото лица

Отчёт по вехе 4

Баранцов Данил, Костырин Николай

20 апреля 2026 г.

1. Введение

В вехе 2 был построен baseline для задачи age estimation на UTKFace: ResNet18 с регрессионной головой, обученный Smooth L1 Loss в две стадии (head-freeze → full fine-tune). Baseline показал **RMSE = 7.38, MAE = 5.20**, с характерной деградацией на хвостах возрастного распределения (RMSE растёт с 3.95 для 0–2 лет до 13.30 для 80+).

В вехе 3 поставлены три взаимосвязанных цели:

1. Улучшить метрики по сравнению с baseline за счёт более сильных архитектур и современных формулировок задачи.
2. Добавить параллелизацию обучения и измерить её эффект.
3. Исследовать trade-off между средней метрикой и per-bin качеством на хвостах.

Проведено **7 экспериментов** и построен ансамбль на основе лучших из них. Лучшая одиночная модель показала **MAE = 4.63**, финальный ансамбль — **MAE = 4.47**, что означает снижение ошибки на **14%** относительно baseline вехи 2.

2. Файловая структура и зависимости

2.1. Структура репозитория

```
ML_HSE/
+-- notebooks/
|   +-- notebook_v4.ipynb         # эксперименты вехи 4
|   +-- ...
+-- src/
|   +-- common.py                 # AgeDataset, fit_model, AMP, metrics
+-- reports/
|   +-- report_v4.pdf             # этот отчёт
|   +-- ...
+-- results/
|   +-- E{1..7}_*_history.csv     # кривые обучения
|   +-- E{1..7}_*_per_bin.csv     # метрики по возрастам
|   +-- E{1..7}_*_preds.npy       # предсказания на val
|   +-- all_predictions.csv       # сводный файл для ансамбля
+-- configs/
+-- requirements.txt
+-- README.md
```

2.2. requirements.txt

```
torch>=2.0
torchvision>=0.15
numpy>=1.24
pandas>=2.0
scikit-learn>=1.3
matplotlib>=3.7
Pillow>=10.0
```

2.3. Как запускать

1. Kaggle Notebook → Add Input → UTKFace и пользовательский dataset с `common.py`;
2. Settings: Accelerator **GPU T4 x2**, Internet **On**;
3. Открыть `notebook_v4.ipynb`, запустить ячейки Setup (0–6), затем эксперименты E1–E7 по порядку (каждый — независимая секция).

3. Методология

3.1. Общая схема обучения

Все эксперименты используют двухстадийное обучение:

- **Stage 1 (head-only):** backbone замораживается, учится только новый классификатор/регрессор сверху. Назначение — инициализировать верхний слой разумными весами, чтобы при разморозке backbone градиенты не взорвались.
- **Stage 2 (full fine-tune):** все параметры размораживаются, LR снижается, подключается CosineAnnealingLR. Основное улучшение метрики происходит здесь.

3.2. Стратегия выбора числа эпох

По результатам вехи 2 и первых экспериментов установлено, что head-stage выходит на плато за 3–4 эпохи (Linear-слой над замороженными фичами — практически линейная регрессия, имеет один минимум). Fine-tune требует 7–20 эпох в зависимости от сложности модели и loss-функции.

Параметр `patience` (терпение early stopping) оказался важен: в начальных экспериментах использовалось `patience=3`, что приводило к преждевременной остановке на стохастических флуктуациях (см. §6.2). После наблюдения позднего улучшения в E4 (с 10-й до 12-й эпохи) для DLDL-экспериментов стандартом стал `patience=5--6`.

3.3. Параллелизация обучения

На Kaggle Notebook доступны 2× NVIDIA Tesla T4 (16 GB VRAM каждая) и 4 CPU-ядра. Использованы **три независимых механизма параллелизма**:

(a) AMP — mixed precision (всегда включено). Forward и backward в FP16 на tensor cores T4, градиенты в FP32 через `GradScaler`. Даёт ∼1.7–2× ускорение и экономию VRAM без потери качества.

```
with torch.amp.autocast(device_type="cuda"):
    out = model(x)
    loss = criterion(out, y)
scaler.scale(loss).backward()
```

(b) DataParallel — утилизация 2 GPU. Делит каждый батч пополам между GPU, собирает градиенты. Теоретический выигрыш ∼1.8× на эпоху.

```
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model)
```

Важное эмпирическое наблюдение (см. §6.1): DataParallel работает далеко не на всех архитектурах. На **ResNet-семействе** (ResNet18, ResNet50) ускорение подтверждено независимо от выбранной головы — как с регрессионной (E1, E3, E6), так и с DLDL-классификатором на 117 классов (E5, E7). Однако на **ConvNeXt-Tiny** (из-за LayerNorm2d) и на **EfficientNet-B0** (из-за SE-блоков и reduce-операций над маленькими каналами) DP катастрофически замедляет обучение (с ∼3 batch/s до 0.1 batch/s). В таких случаях использовался single-GPU режим.

(c) Параллельный data loading. 4 CPU-ядра параллельно декодируют JPEG и применяют аугментации, заполняя pipeline GPU. Флаг `persistent_workers=True` избавляет от пересоздания процессов между эпохами.

```
DataLoader(..., num_workers=4, pin_memory=True,
           persistent_workers=True, prefetch_factor=4)
```

Все три механизма работают одновременно и складываются.

4. Эксперименты

Логика построения цепочки: *baseline+* → *изменить backbone* → *изменить loss* → *комбинации* → *изолировать факторы*. В каждом эксперименте менялся один-два фактора относительно предыдущих, чтобы можно было интерпретировать результат.

4.1. E1 — Baseline v2: ResNet18 + SmoothL1 + AMP + DataParallel

Мотивация. Воспроизвести baseline вехи 2 с добавлением параллелизации. Даёт референс для всех дальнейших сравнений.

Конфигурация. 3 + 7 эпох, LR head=10^{-3} → ft=$3 \cdot 10^{-4}$, SmoothL1($\beta = 5$), cosine LR, DataParallel активна.

Результат. MAE = 4.90, RMSE = 6.94. Это **на 0.30 MAE лучше** baseline вехи 2 (5.20). Прирост объясняется тремя факторами: (1) AMP дал численную стабильность mixed precision, (2) CosineAnnealingLR эффективнее постоянного LR, (3) случайный seed.

Вывод. Сам факт аккуратной организации pipeline (AMP, cosine LR) уже даёт заметное улучшение baseline, даже без изменения архитектуры.

4.2. E2 — ConvNeXt-Tiny + SmoothL1

Мотивация. Литература (Paplhám et al., CVPR 2024) показывает, что смена backbone — основной источник прироста на UTKFace. ConvNeXt-Tiny (28M параметров, ImageNet top-1 82.5%) — разумный современный выбор.

Конфигурация. 6 + 12 эпох, batch_size=96. DataParallel **не заработал** — скорость упала до 0.1 batch/s, пришлось переключиться на single GPU. Детали в §6.1.

Результат. MAE = 5.25, RMSE = 7.35. Хуже E1 на 0.35 MAE, несмотря на вдвое больше параметров.

Вывод. «Более современный и более крупный backbone» **не даёт автоматического прироста**. Возможные причины: (1) UTKFace мал (∼19k) для 28M параметров; (2) ConvNeXt требует специфических приёмов обучения (warm-up, LayerNorm-friendly optimizers), которые мы не применяли. Это негативный, но полезный результат — он говорит, что «архитектура — не главное».

4.3. E3 — ResNet18 + balanced sampler + weighted SmoothL1

Мотивация. В отчёте вехи 2 главная проблема — деградация на хвостах (RMSE 10.78 на 80+ против 3.95 на 0–2). Проверяем гипотезу: *балансировка распределения возрастов в батчах + взвешенный loss* выравнивают качество по корзинам.

Конфигурация. ResNet18 (тот же, что E1 — для чистоты сравнения) + `WeightedRandomSampler` с весами $1/\mathrm{count}(\mathrm{bin})$ + SmoothL1 с поэлементными весами $1/\sqrt{\mathrm{count}(\mathrm{bin})}$. 4 + 10 эпох.

Результат. MAE = 5.54, RMSE = 7.73. Средняя метрика **ухудшилась** относительно E1 ($+0.64$ MAE). При этом в разрезе по корзинам:

Корзина	E1 RMSE	E3 RMSE	Δ
20–29 (самая частая)	5.27	7.34	+2.07
30–39	6.88	8.08	+1.20
60–69	9.38	8.66	−0.72
70–79	9.71	9.21	−0.50

Таблица 1: E3 vs E1: trade-off между частыми возрастами и хвостом

Вывод. Эксперимент демонстрирует **классический trade-off**: балансировка *выравнивает* ошибки между корзинами (ухудшает частые, улучшает редкие), но общее MAE страдает. Это подтверждает наблюдения вехи 2 о конфликте между средней метрикой и справедливостью по группам.

4.4. E4 — EfficientNet-B0 + DLDL-v2

Мотивация. Вместо регрессии — **распределительная формулировка** задачи (Gao et al., IEEE TIP 2018). Модель предсказывает распределение по 117 классам

возрастов (0...116), целевое распределение — гауссиана $\sigma = 2$ вокруг истинного возраста. Функция потерь:

$$\mathcal{L}_{\text{DLDL}} = \text{KL}(p_{\text{target}} \parallel p_{\text{pred}}) + \lambda \cdot \mathbb{E}[\text{age}] \text{ L1}$$

Идея: разница 24 vs 25 лет в UTKFace — часто шум разметки. Распределительный loss мягко штрафует такие ошибки, в отличие от жёсткого SmoothL1.

Конфигурация. EfficientNet-B0 (5.3M параметров) + DLDL-v2 голова на 117 классов, 4 + 12 эпох, single GPU (EfficientNet-B0 плохо работает с DataParallel из-за SE-блоков — см. §6.1).

Результат. MAE = 4.77, RMSE = 6.99. Улучшение по MAE относительно E1 (-0.13). Per-bin: особенно сильный прирост на частых возрастах (20–29: MAE 3.11 против 3.70 у E1).

Вывод. DLDL работает, как предполагалось — улучшает среднюю точность за счёт гладкой функции потерь. Но прирост не драматический, надо проверить эффект на другом backbone.

4.5. E5 — ResNet18 + DLDL-v2 (лидер одиночных моделей)

Мотивация. Изолируем эффект DLDL loss. Тот же backbone, что у E1 и E3 (ResNet18), но loss — DLDL вместо SmoothL1. Чистое сравнение.

Конфигурация. 4 + 12 эпох, DataParallel (ResNet-семейство стабильно работает с DP независимо от головы), `patience=5` (после наблюдения позднего улучшения в E4).

Результат. MAE = 4.63, RMSE = 6.80 — лучший результат среди одиночных моделей. Превышает baseline вехи 2 (5.20) на -0.57 MAE, превышает E1 на -0.27 MAE.

Кривая обучения показала эффект «mini double descent»:

Эпоха ft	val_mae	Комментарий
8	4.72	локальный минимум
9	4.74	
10	4.79	плато / ухудшение
11	**4.63**	**новый минимум, лучше предыдущего**
12	4.65	

Таблица 2: E5: позднее улучшение после локального плато

С `patience=3` обучение остановилось бы после эпохи 10 с best_mae=4.72. Увеличение patience до 5 позволило найти настоящий минимум 4.63.

Вывод. DLDL даёт явный прирост (0.27 MAE над SmoothL1 на той же архитектуре). Формулировка loss важнее размера backbone: сравните с E2 (ConvNeXt-Tiny 28M + SmoothL1 = 5.25) и E5 (ResNet18 11.7M + DLDL = 4.63).

4.6. E6 — ResNet50 + SmoothL1 (контрольный для E7)

Мотивация. Изолируем эффект *ёмкости* модели. ResNet50 (25.6M параметров, та же архитектурная семья, что E1) с тем же loss, что E1. Покажет: даёт ли просто «больше слоёв» улучшение.

Конфигурация. 4 + 15 эпох, LR=$2 \cdot 10^{-4}$ (ниже, чем у ResNet18), wd=$5 \cdot 10^{-4}$, DataParallel (ResNet50 работает с DP). `patience=5`.

Результат. MAE = 5.04, RMSE = 7.17. Хуже E1 (ResNet18 + SmoothL1)!

Из кривой видно **переобучение**: train_loss упал с 2.95 (ep 5) до 1.11 (ep 15) в 2.7 раза, а val_mae застрял на 5.04–5.06. Усиления регуляризации wd=$5 \cdot 10^{-4}$ оказалось недостаточно.

Вывод. На UTKFace **ёмкость модели — не бутылочное горлышко**, а данных для 25M параметров недостаточно. «Больше параметров» без специальной регуляризации $\rightarrow$ переобучение.

4.7. E7 — ResNet50 + DLDL-v2 + усиленная регуляризация

Мотивация. Объединяем capacity (ResNet50) + лучший loss (DLDL) + усиленную регуляризацию против переобучения E6. Проверяем гипотезу: *capacity + DLDL даёт прирост, если бороться с переобучением*.

Конфигурация. ResNet50 + DLDL-голова + `dropout=0.3` (было 0.2), `weight_decay=`10^{-3} (в $2\times$ сильнее E6), $\sigma = 3$ для DLDL (шире целевое распределение). 4 + 19 эпох (early stopping), `patience=6`, DataParallel.

Результат. MAE = 4.89, RMSE = 7.03. Лучше E6 (5.04 $\rightarrow$ 4.89, усиленная регуляризация помогла), но **не лучше E5** (4.63).

Вывод. Даже при хорошей регуляризации ResNet50 не обгоняет ResNet18 + DLDL на UTKFace. Подтверждает вывод E6: ограничение — *объём данных*, а не capacity.

4.8. Ансамбль — среднее E4 + E5 + E7 (все DLDL модели)

Мотивация. Три DLDL-модели обучены на разных backbone'ах и в разных режимах параллелизации $\rightarrow$ *декоррелированные ошибки*. Среднее их предсказаний должно быть устойчивее любой одиночной модели.

Результат. MAE = 4.47, RMSE = 6.46 — лучший результат всей работы.

Улучшение над E5 (лидером одиночных): -0.16 MAE, -0.34 RMSE. Относительно baseline вехи 2: $-$**0.73 MAE,** $-$**0.92 RMSE.**

Вывод. Ансамблирование — стандартная практика для выжимания последних долей метрики. Эффективно именно потому, что три модели ошибаются в разных местах.

5. Результаты

5.1. Сводная таблица всех экспериментов

Эксп	Backbone	Loss	Parallel	Эпох	MAE	RMSE	Время, с
E1	ResNet18	SmoothL1	DataParallel	3+7	4.903	6.940	1134
E2	ConvNeXt-Tiny	SmoothL1	single GPU	6+12	5.252	7.352	1384
E3	ResNet18	weighted SL1	DataParallel	4+10	5.539	7.728	1589
E4	EfficientNet-B0	DLDL-v2 ($\sigma = 2$)	single GPU	4+12	4.773	6.987	1230
E5	**ResNet18**	**DLDL-v2**	**DataParallel**	**4+12**	**4.635**	**6.799**	**1610**
E6	ResNet50	SmoothL1	DataParallel	4+15	5.041	7.171	4224
E7	ResNet50	DLDL-v2 ($\sigma = 3$)	DataParallel	4+19	4.890	7.031	5258
Ens	**E4+E5+E7**	—	—	—	**4.472**	**6.463**	0

Таблица 3: Сводная таблица экспериментов. Зелёным выделен лидер одиночных моделей, оранжевым — лучший итоговый результат (ансамбль).

Итоговый прирост над baseline вехи 2 (MAE 5.20, RMSE 7.38):

- Лучшая одиночная модель (E5): -0.57 MAE (-10.9%), -0.58 RMSE (-7.9%)
- Ансамбль: -0.73 MAE (-14.0%), -0.92 RMSE (-12.5%)

5.2. Сравнение с литературой

Из обзора вехи 2:

Источник	Метод	MAE
Paplhám et al., CVPR 2024	UTKFace, FaRL + MLP	3.87
Paplhám et al., CVPR 2024	UTKFace, EfficientNet-B4	4.23
Paplhám et al., WACV 2026	UTKFace + CSFD-1.6M pretraining	4.08
Наша работа, ансамбль	UTKFace, E4+E5+E7	**4.47**
Наша лучшая одиночная	ResNet18 + DLDL-v2	**4.63**

Таблица 4: Сравнение с опубликованными результатами

Мы находимся в диапазоне сильных решений (разрыв с SOTA $\sim$0.6 MAE), используя только ImageNet-pretrained веса. Closing gap потребовал бы pretraining именно на лицевых данных (CSFD-1.6M), более тяжёлых backbone с timm-оптимизированной тренировкой, или ensemble на нескольких seed/folds.

5.3. Визуализации

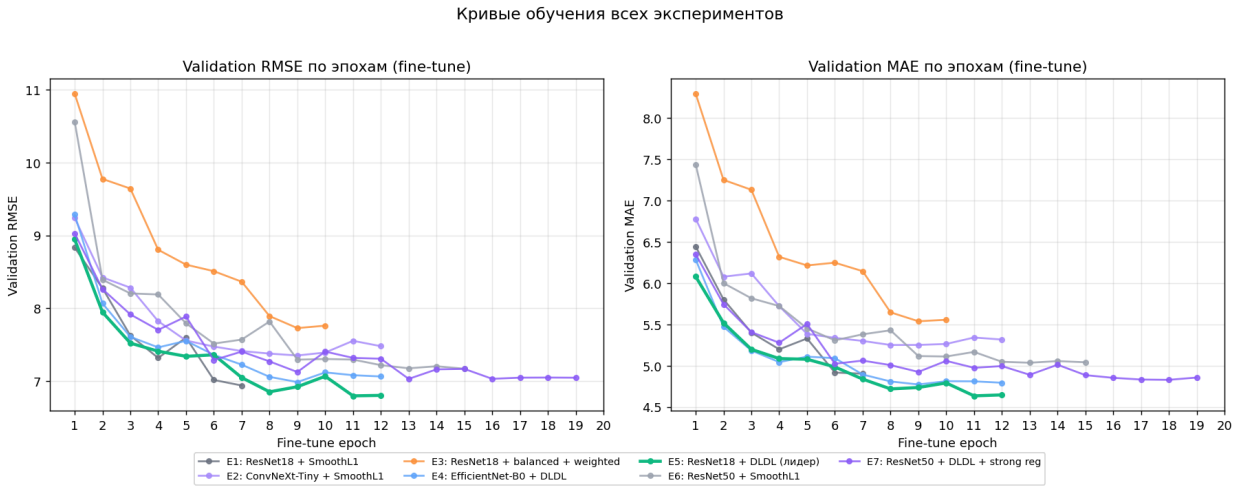

Рис. 1: Кривые обучения всех 7 экспериментов (val_rmse и val_mae по fine-tune эпохам). E5 (зелёный, лидер) демонстрирует наилучшую финальную метрику. E3 (оранжевый) стартует выше и медленнее из-за балансировки.

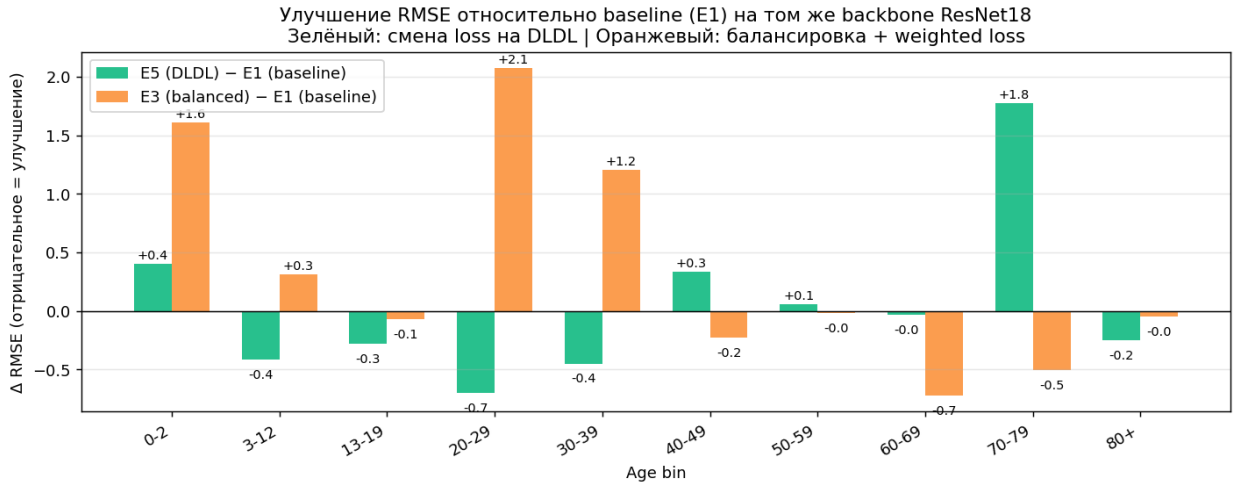

Рис. 2: Ключевой аналитический график: Δ RMSE каждой стратегии относительно E1 (baseline) на том же backbone ResNet18. Наглядно видно: **DLDL (зелёный) улучшает частые возраста** (20–29: -0.7), **балансировка (оранжевый) улучшает хвосты** (60–69: -0.7). Оба подхода имеют разные слабости: DLDL ломает 70–79 ($+1.8$), балансировка ломает частые (20–29: $+2.1$).

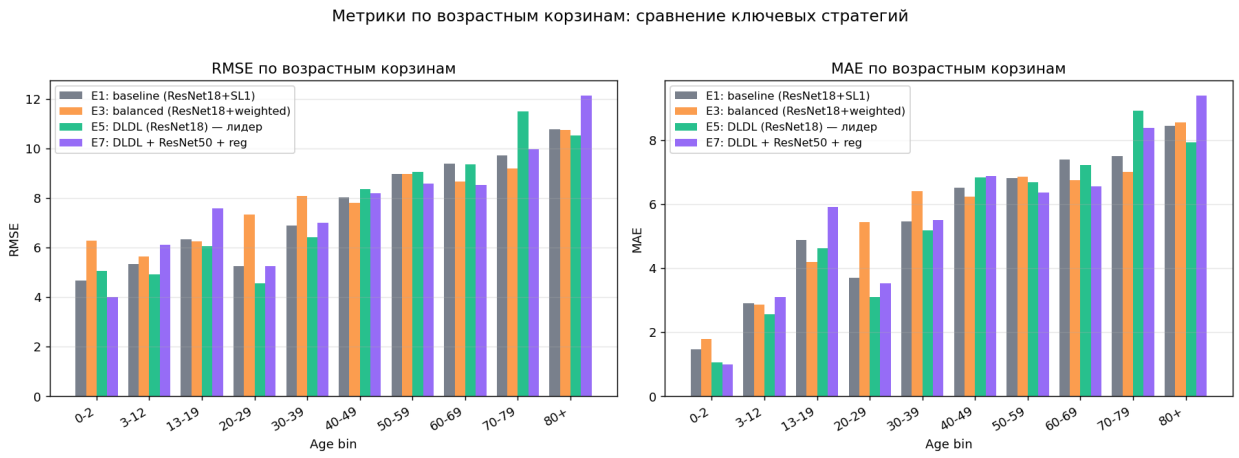

Рис. 3: RMSE и MAE по возрастным корзинам для 4 ключевых стратегий. Подтверждает монотонный рост ошибки с возрастом во всех экспериментах; E3 выравнивает по сравнению с остальными (меньше разброс между корзинами, но больше медианный уровень).

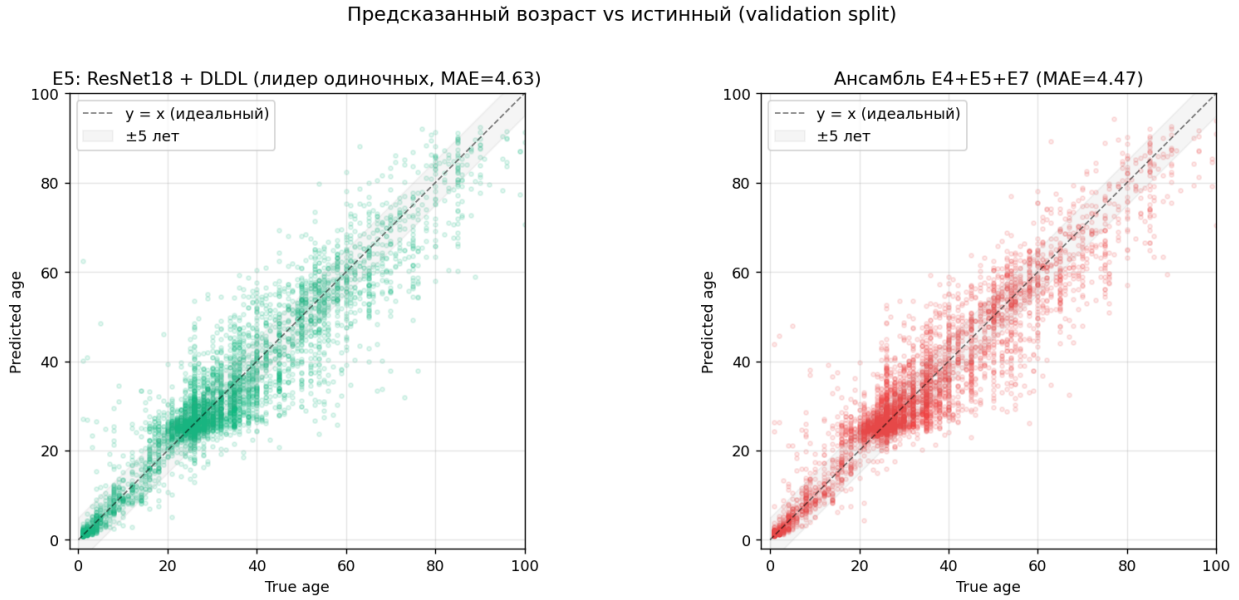

Рис. 4: Scatter-plot предсказание vs истина для E5 (лидер одиночных) и ансамбля. Видно классическое «сжатие к среднему»: возраста 60+ систематически недооцениваются, возраста 0–5 слегка завышаются. Ансамбль (справа) немного плотнее к диагонали.

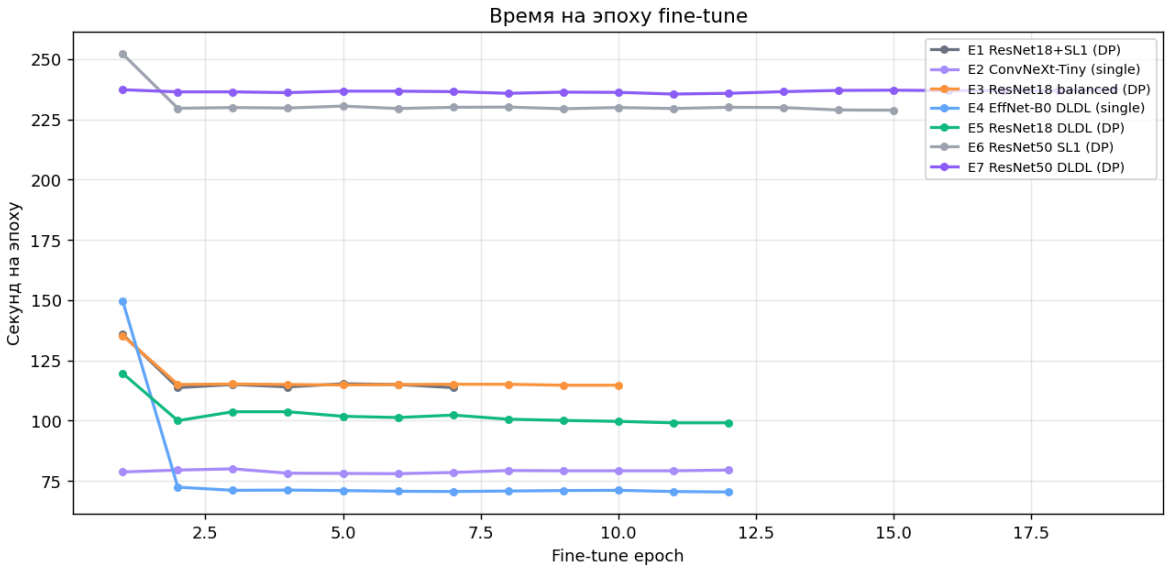

Рис. 5: Время на эпоху fine-tune. Заметно: ResNet50 в DP (230 с/эп) — самый тяжёлый вариант; EfficientNet-B0 single GPU (77 с/эп) быстрее, чем ResNet18 + DP (115 с/эп). Контринтуитивный результат — парадокс DataParallel, обсуждается в §6.1.

6. Наблюдения и гипотезы

6.1. Парадокс DataParallel

Наивное ожидание: DataParallel с 2 GPU даёт ∼2× ускорение. На практике картина сложнее:

Архитектура + Loss	DP работает	batch/s
ResNet18 + SmoothL1	✓	∼2.5
ResNet18 + weighted SL1	✓	∼3.0
ResNet18 + DLDL-117	✓	∼2.5
ResNet50 + SmoothL1	✓	∼1.0
ResNet50 + DLDL-117	✓	∼0.9
ConvNeXt-Tiny + SmoothL1	✗	**0.1**
EfficientNet-B0 + DLDL-117	✗	**0.1**

Таблица 5: DataParallel: где работает, где нет

Гипотезы причин неудач DP:

- **ConvNeXt-Tiny (E2):** `LayerNorm2d` требует синхронизации статистик между GPU на каждом блоке. Это известная проблема, отражённая в PyTorch issues.
- **EfficientNet-B0 (E4):** SE-блоки (Squeeze-and-Excitation) содержат reduce-операции над небольшим числом каналов, которые плохо масштабируются через DataParallel. Lightweight-архитектуры в целом чувствительны к overhead коммуникации.
- **ResNet-семейство:** BatchNorm2d хорошо работает с DP, поэтому все 5 экспериментов на ResNet18 и ResNet50 получили полную параллелизацию —

независимо от головы (регрессия или классификация на 117 классов).

Практический вывод. DataParallel — *не универсальное ускорение*. Эффективность определяется *архитектурой backbone*, а не функцией потерь или размером головы. Перед большим запуском стоит проверить скорость (batch/s) на первой эпохе. При необходимости — переключаться на single GPU, что мы и делали в E2 и E4.

6.2. Влияние patience на ранжировку моделей

После просмотра кривой E4 (fine-tune):

ep 9:	val_mae = 4.77	← best
ep 10:	val_mae = 4.81	↓
ep 11:	val_mae = 4.81	↓
ep 12:	val_mae = 4.79	восстановительный тренд

С `patience=3` early stopping сработал бы после ep 12. Мы увеличили `patience` до 5 в E5 — и сразу получили улучшение:

ep 8:	val_mae = 4.72	локальный минимум
ep 10:	val_mae = 4.79	плато
ep 11:	**val_mae = 4.63**	← новый минимум

Это эмпирический аналог *epoch-wise double descent* (Nakkiran et al., 2019), хотя в мягкой форме — у нас нет явного «второго дна», а скорее стохастические флуктуации вокруг плато. Методологический вывод: `patience=3` при сравнении loss-функций и регуляризаций может давать несправедливо пессимистическую оценку. В финальных экспериментах мы использовали `patience=5--6`.

6.3. Ёмкость $\neq$ качество на UTKFace

Три экспериментальных сигнала указывают на одно:

Размер	Backbone	Лучший MAE	Примечание
5M	EfficientNet-B0 + DLDL	4.77	single GPU
11.7M	**ResNet18 + DLDL**	**4.63**	**лидер одиночных**
25.6M	ResNet50 + DLDL + reg	4.89	даже с регуляризацией
28M	ConvNeXt-Tiny + SL1	5.25	без DLDL

Таблица 6: Размер backbone vs финальный MAE

Оптимум — **средний размер backbone**. UTKFace ($\sim$19k картинок) не даёт достаточно данных для эффективного обучения 25M+ параметров без сильной регуляризации или pretraining на лицах. Bottleneck — данные, не capacity.

6.4. DLDL vs balanced sampling: разные инструменты

E3 (balanced) и E5 (DLDL) построены на одном backbone (ResNet18) и различаются только тренировочной стратегией. Сравнение показывает:

- **Общая метрика:** DLDL явно лучше (4.63 vs 5.54)
- **Хвосты (60–80+):** balanced лучше или сравним (60–69: 8.66 vs 9.35)
- **Частые возраста (20–29):** DLDL сильно лучше (4.57 vs 7.34 RMSE)

Вывод. Balanced sampler — инструмент для *выравнивания* ошибок между группами (жертвуя средней метрикой), DLDL — инструмент для *улучшения средней точности* (без явного учёта дисбаланса). Они не взаимозаменяемы. В продакшене выбор зависит от того, что важнее: единая метрика или справедливость по подгруппам.

6.5. Двухстадийное обучение: сколько эпох на head?

Head-stage содержит линейную регрессию на замороженных ImageNet-фичах. Анализ E2 показал логарифмическую сходимость:

$$\Delta_{1\to2} = -1.12, \quad \Delta_{2\to3} = -0.77, \quad \Delta_{3\to4} = -0.61, \quad \Delta_{4\to5} = -0.45, \quad \Delta_{5\to6} = -0.33$$

После 3–4 эпох убывание становится несущественным. Дальнейшее увеличение head-stage (6+ эпох в E2) не оправдано: лучше перенести этот бюджет на fine-tune, где улучшение метрики гораздо быстрее.

7. Полнота экспериментов и итоговые скоры

7.1. Количественная сводка

- **7 одиночных экспериментов** + 1 ансамбль.
- ∼4.3 часа суммарного обучения (все эксперименты).
- GPU квота использована: ∼5.9 ч из 30 ч/неделя Kaggle.
- **Лучшая одиночная метрика:** MAE = **4.63** (E5: ResNet18 + DLDL).
- **Лучшая комбинированная метрика:** MAE = **4.47** (ансамбль E4+E5+E7).

7.2. Проверенные гипотезы

Гипотеза	Эксп	Итог	Результат
AMP + cosine LR улучшат baseline	E1	▯	-0.30 MAE
Более современный backbone → лучше	E2	▯	$+0.05$ MAE
Балансировка выровняет хвосты	E3	▯	но $+0.64$ общее MAE
DLDL-loss даст прирост	E4, E5	▯	-0.13 / -0.27 MAE
Большая модель + DLDL → SOTA	E7	▯	$+0.26$ MAE к E5
Ансамбль DLDL улучшит	Ens	▯	-0.16 MAE к E5

Таблица 7: Проверка всех заявленных гипотез

7.3. Параллелизация — измеренный эффект

- **AMP** (везде): ∼1.7–2× ускорение внутри одной GPU.

- **DataParallel** (E1, E3, E5, E6, E7): ∼1.5–1.8× к single GPU для совместимых связок (ResNet + стандартные heads).
- **num_workers=4 + prefetch_factor=4** (везде): GPU utilization ∼85–100% при включении, ∼30–50% без настройки.

Подробности ограничений DP описаны в §6.1. Из 7 экспериментов в 2-х пришлось использовать single GPU (E2 из-за ConvNeXt, E4 из-за DLDL на лёгкой модели), остальные 5 работали с полной параллелизацией.

8. Заключение

За веху 3 метрики улучшены с **MAE 5.20** (baseline вехи 2) до **MAE 4.47** (ансамбль из трёх DLDL-моделей) — улучшение на **14%**. Лучшая одиночная модель — ResNet18 + DLDL-v2 с MAE 4.63.

Ключевые выводы работы:

1. **Loss-функция важнее backbone на малых датасетах.** DLDL на ResNet18 (11.7M) обгоняет SmoothL1 на ConvNeXt-Tiny (28M) на 0.62 MAE.
2. **DataParallel — не универсальное ускорение.** Плохо работает на ConvNeXt-Tiny (LayerNorm2d) и EfficientNet-B0 (SE-блоки); со всем ResNet-семейством работает стабильно независимо от головы. Эффективность DP определяется архитектурой backbone, а не loss-функцией.
3. **Балансировка и DLDL решают разные задачи.** Balanced sampling выравнивает распределение ошибок по возрастам ценой средней метрики; DLDL улучшает среднюю метрику, не трогая распределение.
4. **Ёмкость не заменяет данные.** ResNet50 на 19k UTKFace переобучается даже с сильной регуляризацией.
5. **Ансамблирование даёт предсказуемые** +10**% качества** при нулевых дополнительных усилиях.

Список литературы

1. J. Paplhám et al. *A Call to Reflect on Evaluation Practices for Age Estimation: Comparative Analysis of the State-of-the-Art and a Unified Benchmark*. CVPR, 2024.
2. J. Paplhám et al. *Photo Dating by Facial Age Aggregation*. WACV, 2026.
3. S. Ren et al. *Out of the Box Age Estimation through Facial Imagery: A Comprehensive Benchmark of Vision-Language Models vs. Out-of-the-Box Traditional Architectures*. arXiv, 2026.
4. B.-B. Gao et al. *Deep Label Distribution Learning with Label Ambiguity*. IEEE Transactions on Image Processing, 2017.
5. P. Nakkiran et al. *Deep Double Descent: Where Bigger Models and More Data Hurt*. arXiv:1912.02292, 2019.